

Technical Requirements Document (TRD)

AI-Powered Automatic Block Planning System for Indian Railways

Prototype / MVP Technical Specification for Antigravity Development

1. Purpose

This TRD defines the technical architecture, data model, application modules, AI/decision logic, APIs, security, deployment and acceptance requirements for the MVP. The prototype integrates TMS, SMMS, TDMS, BDMS and COA data without replacing those systems.

2. Technical Objectives

- Import prototype CSV datasets into Supabase PostgreSQL.
- Create a Data Hub that validates, cleans and standardizes departmental records.
- Unify maintenance records using common identifiers such as maintenance ID, date and KM location.
- Calculate explainable maintenance priority using current and historical factors.
- Identify nearby maintenance activities that can potentially be grouped.
- Analyze COA passenger/goods train schedules and maintenance block windows.
- Detect schedule conflicts and recommend low-impact block windows.
- Provide a dashboard and human approval workflow.

3. High-Level Architecture

Frontend → Application/API Layer → Data Hub → Supabase PostgreSQL → AI Priority & Block Planning Engine → Recommendations → Officer Approval

The system should be modular so the prototype can later be connected to authorized live railway systems.

4. Recommended Technology Stack

Layer	Recommended MVP technology
Frontend	React / Next.js with responsive dashboard UI
Backend/API	Python FastAPI or Node.js API
Database	Supabase PostgreSQL
Authentication	Supabase Auth
Data Processing	Python + Pandas
AI/Decision Engine	Python rule-based/weighted scoring for MVP; ML-ready design
Charts	A React charting library
Deployment	Antigravity-supported web deployment + Supabase cloud

5. Data Sources and Storage

Create separate raw/source tables so the original departmental data remains traceable.

Table	Key content
tms_data	Track defects, maintenance, severity, KM location, duration and history
smms_data	Signal/telecommunication defects, maintenance and history
tdms_data	Traction/OHE/electrical defects, maintenance and history
bdms_data	Block requests, block requirements, postponement/disconnection information
coa_data	Train movements, passenger/goods type, timings, traffic and block windows
unified_maintenance	Standardized records consumed by AI
priority_results	Priority score, priority class and explanation
block_recommendations	Grouped tasks, window, conflicts, score and explanation
approvals	Officer action, timestamp and comments

6. Common Data Fields

The main cross-system join fields for the prototype should be **location in KM, date, maintenance ID/request ID where available, and time window**. A section field is optional and should not be required if it is absent from the existing source CSVs.

Recommended unified fields: maintenance_id, source_system, department, asset_id, location_km, defect_type, severity, status, block_required, estimated_duration_min, historical_actual_duration_min, overdue_days, occurrence_count, postponement_count, last_maintenance_date, maintenance_date.

7. Data Hub

- CSV ingestion and schema validation.
- Normalize column names and severity values.
- Convert KM values to numeric form for proximity calculations.
- Normalize dates and times to a common format.
- Flag missing required fields and duplicate records.
- Preserve source_system and source_record_id for traceability.
- Create/update unified_maintenance records.
- Expose clean records to the AI services.

8. Priority Engine

For the MVP, use an explainable weighted scoring model rather than claiming a trained ML model without sufficient real historical data. Example configurable factors:

Factor	Example influence
Current severity	Critical > High > Medium > Low
Overdue days	Higher overdue duration increases priority
Asset criticality	Safety/operationally critical assets increase priority
Repeat occurrence	Repeated defects increase priority
Postponement risk	Repeated postponements increase priority

Historical severity	Repeated high/critical history increases priority
Historical duration	Used for realistic block sizing, not priority alone

Output: priority_score (0–100), priority_class, factor_breakdown and explainable_note.

9. Nearby Maintenance Grouping

Convert KM locations to numeric values and calculate distance between compatible tasks. For example, a configurable threshold such as 0.5–1.0 km can be used for the prototype. Grouping must also consider department requirements, block type, compatibility and combined duration.

The grouping engine should return group_id, task IDs, KM range, combined duration, required block type and grouping reason.

10. COA Conflict and Block Engine

- Compare proposed maintenance interval with scheduled train intervals.
- Identify passenger and goods trains affected by a proposed block.
- Give passenger movements higher operational importance for the prototype impact score.
- Check whether a maintenance group fits inside an available block window.
- Add buffer time where configured.
- Reject or penalize windows with unacceptable conflicts.
- Rank feasible windows by operational impact, available duration and maintenance priority.

11. Block Recommendation Output

Each recommendation should contain: recommendation_id, maintenance_group_id, date, block_start, block_end, total_duration, affected_train_count, passenger_conflict_count, goods_conflict_count, feasibility_status, recommendation_score, explanation, created_at and approval_status.

12. Explainable AI

Every result must be understandable without inspecting model internals. Example:

Priority: Critical. Critical defect + overdue work + repeated historical occurrences + postponement risk contributed to the score.

Block: 23:30–02:30. The window has no scheduled train conflict, provides sufficient duration based on historical maintenance time, and allows nearby compatible tasks to be grouped.

13. API Requirements

Method	Endpoint	Purpose
GET	/maintenance	List/filter unified maintenance
GET	/maintenance/{id}	Maintenance details/history
POST	/import/{source}	Import/validate source CSV
POST	/priority/analyze	Calculate/recalculate priority
GET	/priority	View ranked priority results
GET	/coa/schedule	Retrieve COA operational data
POST	/blocks/recommend	Generate block recommendations
GET	/blocks/recommendations	List recommendations
POST	/blocks/{id}/approve	Approve recommendation
POST	/blocks/{id}/reject	Reject recommendation

14. Frontend Requirements

- Dashboard with counts of Critical/High/Medium/Low maintenance.
- Maintenance table with department, KM, severity, overdue days and priority score.
- Historical details panel.
- COA timetable and operational conflict view.
- Map-like KM visualization can be simulated for the MVP.
- Grouped maintenance view showing compatible tasks.
- Block recommendation card with time, duration, conflicts and explanation.
- Approval controls for authorized/simulated officer users.

15. Supabase Requirements

- PostgreSQL tables with primary keys and timestamps.
- Foreign keys where stable identifiers exist.
- Indexes on location_km, maintenance date, severity and status.
- Row-level security for user-facing tables where authentication is enabled.
- Database views for unified maintenance and dashboard summaries.
- Seed data import from the prototype CSV files.

16. Validation and Error Handling

- Reject malformed dates/times and non-numeric KM values.

- Flag missing maintenance IDs or required fields.
- Prevent duplicate import using source_system + source_record_id where available.
- Show warnings rather than silently modifying source data.
- Record import status and validation errors.

17. Security and Safety

This MVP must use synthetic/sample data unless authorized railway data access is explicitly provided. It must not directly control signalling, train movement, traction power or railway field equipment. AI output is advisory only. Final block decisions remain with authorized railway personnel.

- Use authenticated access for protected pages.
- Keep service credentials server-side.
- Do not expose Supabase service-role keys in frontend code.
- Log approval actions and timestamps.

18. Performance Targets for MVP

- Dashboard initial load target: under 3 seconds for sample dataset.
- Priority analysis: under 5 seconds for the prototype dataset.
- Block recommendation generation: under 10 seconds for the prototype dataset.
- CSV validation should provide clear results without requiring manual database editing.

19. Testing Requirements

- Unit tests for severity/priority scoring.
- Tests for KM proximity grouping.
- Tests for train/maintenance time overlap.
- Tests for block-window duration feasibility.
- CSV validation tests for missing and malformed values.
- End-to-end test: CSV → Supabase → Data Hub → Priority → COA conflict → Recommendation → Approval.

20. MVP Acceptance Criteria

- Five source datasets can be imported and viewed.
- Unified maintenance records are generated.
- Priority scores and explanations are produced.
- Historical and postponement factors are visible in decisions.
- Nearby maintenance tasks can be grouped by KM proximity.
- Passenger and goods train conflicts are detected.
- A feasible block window can be recommended.
- Recommendations contain an explanation.
- Officer approval/rejection is recorded.
- No AI recommendation can directly control railway operations.

21. Antigravity Implementation Prompt

Build the MVP described in this TRD as a responsive web application. Use Supabase PostgreSQL as the database and import the existing TMS, SMMS, TDMS, BDMS and COA CSV datasets. Keep raw source tables separate, then create a Data Hub layer that validates and standardizes records into a unified maintenance view. Implement an explainable weighted priority engine using severity, overdue days, asset criticality, repeat occurrences, postponement risk and historical information. Implement KM-based nearby maintenance grouping and a COA conflict engine that compares maintenance windows with passenger and goods train schedules. Generate ranked block recommendations using feasible duration, train conflict impact and maintenance priority. Show the reasons behind every priority and recommendation. Provide dashboard, maintenance, COA, conflict, recommendation and officer approval screens. Keep all outputs advisory and require human approval.

22. Future Technical Extensions

- Real-time authorized system connectors/APIs.
- ML models trained on validated historical railway data.
- Optimization algorithms for multi-task block scheduling.
- GIS corridor visualization.
- Real-time train movement feeds.
- Prediction of maintenance duration and postponement risk.